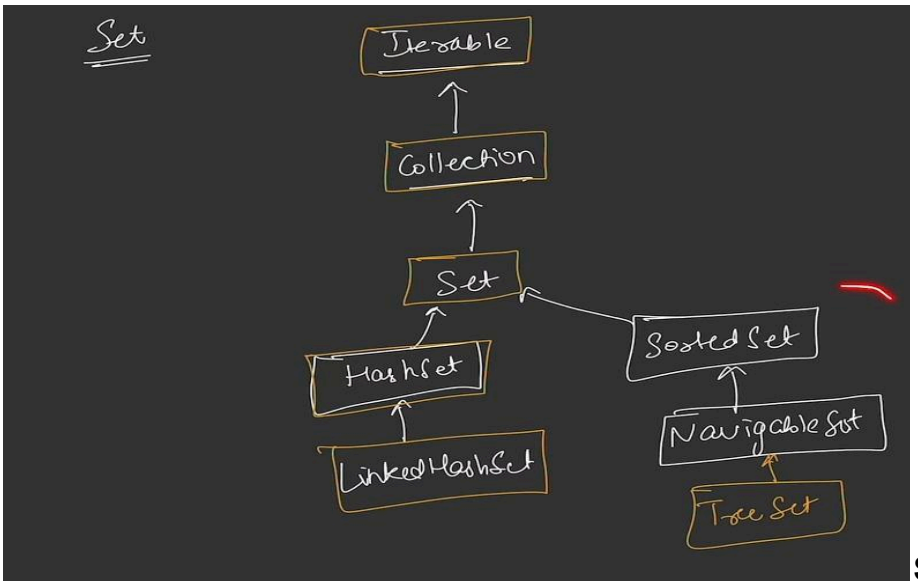
Methods & Cunstroctors in Set & Map

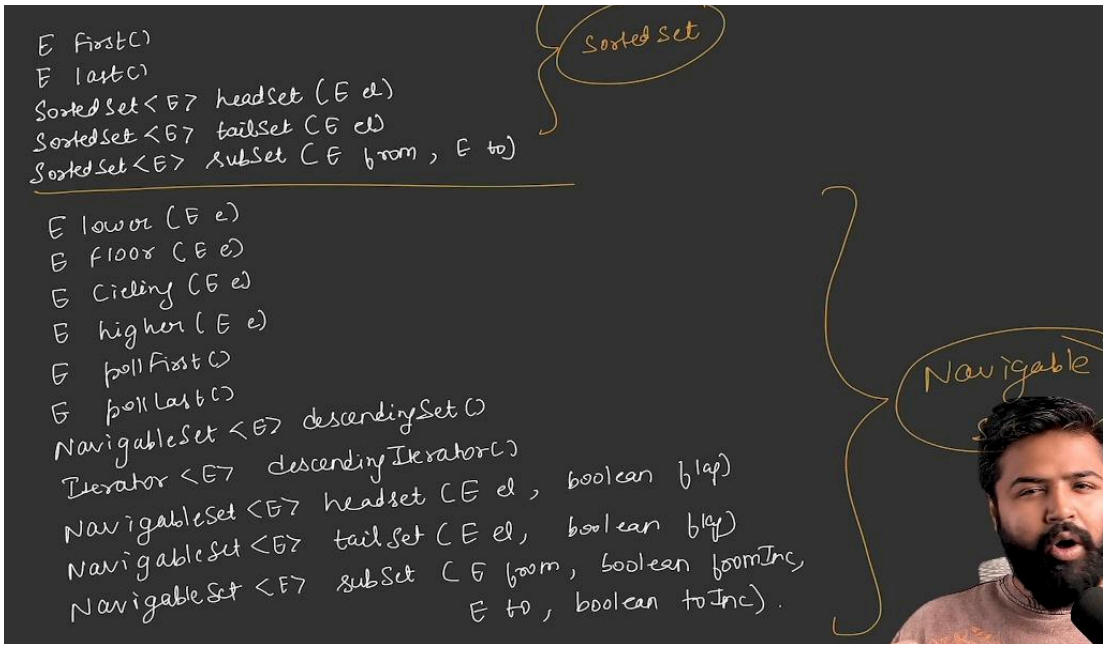


Sorted set & Navigable set are Interfaces.

Cunstroctors of Hash-Set & Linked-HashSet

```
// Constructors of HashSet ,& LinkedHashSet
            Set<Integer> set = new HashSet<>(); // internal size--> 16
        // initial capacity
            Set<Integer> set2 = new HashSet<>(100);
        // capacity, Load factor--> defaults---> 16, 0.75
            Set<Integer> set3 = new HashSet<>(100, 0.8f);
        // using another collection
            Set<Integer> set4 = new HashSet<>(List.of(1,2,3,4,5,6,7,8,9));
    // both do'nt have their methods because they implements collection and iterable interfaces.
```

Methods of TreeSet class




```
// Cunstroctors of TreeSet
        //Set<Integer> set2 = new TreeSet<>(List.of(2,5,7,9));
        TreeSet<Integer> set = new TreeSet<>();
            set.add(80 );
            set.add(23 );
            set.add(10 );
            set.add(90 );
            set.add(50 );
    // Sorted Set Interface's Methods
        System.out.println(set.first()); // to get the smallest value.--> 10
        System.out.println(set.last()); // to get the largest value.--> 90
        System.out.println(set.headSet(80 )); // it gives the smaller no's than 80, 80 is excluded.
        System.out.println(set.tailSet(80));// it gives the 80 and larger then 80.
```

```
        System.out.println(set.subSet(23, 80));//23 is inclusive & 80 is exclusive.
    // Navigable Set extends Sorted Set methods
        System.out.println(set.lower(80));// gives largest no lower than 80.
        System.out.println(set.floor(80));// gives greatest element less then or equal to 80.
        System.out.println(set.higher(80));// smallest no greater than 80.
        System.out.println(set.ceiling(80 ));// smallest no greater than or equal to 80.
        // System.out.println(set.pollFirst());// it gives & removes the smallest no.
        // System.out.println(set.pollLast());// it gives & removes the largest no.
        System.out.println(set.descendingSet());
        Iterator<Integer> it = set.descendingIterator();
            while(it.hasNext()){
                System.out.println(it.next());
            }
        System.out.println(set.headSet(80, true));// to include 80.
        System.out.println(set.tailSet(80, false));// to exclude 80.
        System.out.println(set.subSet(10, false, 80, true));
```

MAP interface methods

Map ko sabhi methods khud se define krne pdete ha kuki ye kisi bhi interface ko impliment nahi krte ha.

```
    Map<Integer, String> map = new HashMap<>();
  // RollNumber, Names
    map.put(101, "Priyansh");
    map.put(102, "Tanuja");
    map.put(103, "Tanveer");
    map.put(104, "Anuja");
    map.put(105, "Tanisha");
        System.out.println(map.size());
        System.out.println(map.isEmpty());
        System.out.println(map.containsKey(107 ));
        System.out.println(map.containsValue("Priyansh"));
        System.out.println(map.get(101));// gives value of the particular key.
        System.out.println(map.put(106, "Raghav"));// ans--> null,
        System.out.println(map.put(103, "Ramesha"));// ans-->Tanveer, but Tanveer is replaced by Ramesha.
        //System.out.println(map.remove(101));
        //map.putAll(map2);// kisi or map ki sabhi values ko put kr skte ha.
        //map.clear();
    //Important Methods of Map--->
        map.keySet();
        Set<Integer> set = map.keySet();
        System.out.println(set);// yha pr map ki sabhi keys ka set array bn jayga.
        map.values();
        Collection<String> c = map.values();
        System.out.println(c );// yha prr map ki sabhi values aa jaygi.
        map.entrySet();
        // Set<Map.Entry<Integer, String>> entries = map.entrySet();
        // System.out.println(entries);// yaha pr key & values dono mil jate ha.
        System.out.println(map.getOrDefault(109, "unknown"));// default value.
        System.out.println(map.putIfAbsent(103, "Abhay"));
        //putIfAbsent does not replace the existing value.
        map.remove(101, "Aditya");// it only removes if there is same match of key,value.
        map.replace(101, "Sonu");//agar key already exist krti hogi to hi value replace hogi.
        map.replace(101, "Tanujing", "RAngbaaz");// only replace agar old value or key match krte ha.
        // Set<Map.Entry<Integer, String>> entries = map.entrySet();
        //     for(Map.Entry<Integer, String> entry : entries){
        //         Integer key = entry.getKey();
        //         String value = entry.getValue();
        //             System.out.println(key + ", " + value);
        //     }
        // Immutable map
        Map<Integer, String> map2 = Map.of(202, "Ramesh", 203, "Arnav" );
        Set<Map.Entry<Integer, String >> entries = map2.entrySet();
            System.out.println(entries);
```

Cunstroctors of HashMap & Linked HashMap

```
// HashMap & LinkedHashMap--> both works same.
            // Cunstroctors
            Map<Integer, String> map = new HashMap<>();
            Map<Integer, String> map2 = new HashMap<>(100);
            Map<Integer, String> map3 = new HashMap<>(100,0.8f);
            Map<Integer, String> map4 = new HashMap<>(map3);
            Map<Integer, String> map5 = new HashMap<>(Map.of(101, "Aditya", 102, "Priyansh"));
```

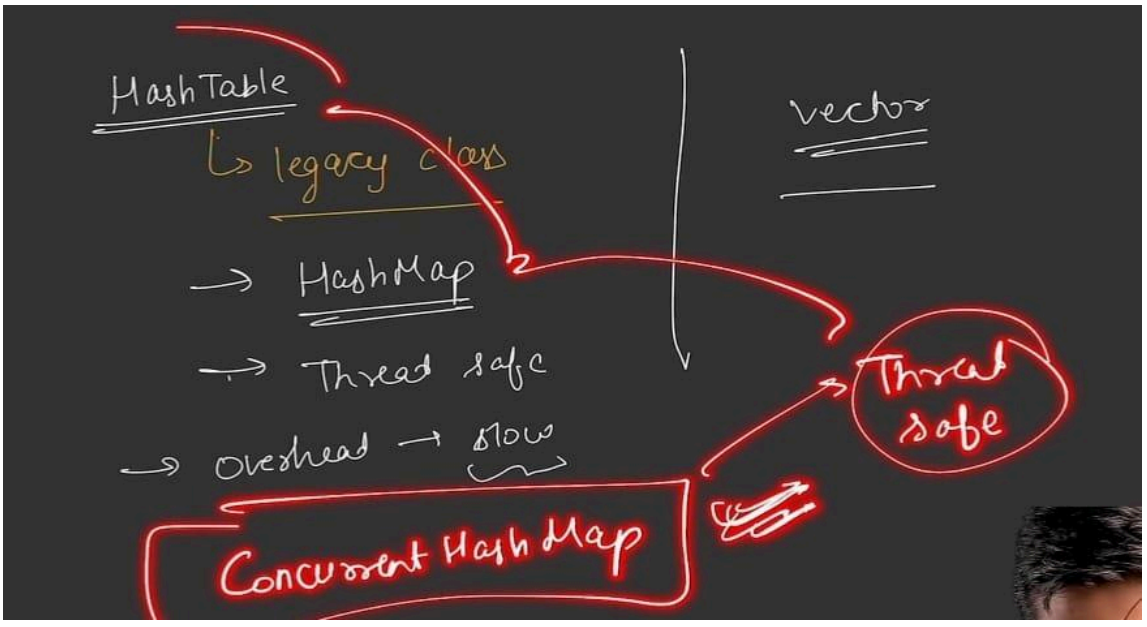
Methods of TreeMap

```
  TreeMap<Integer, String> map = new TreeMap<>();
            map.put(101, "Aditya");
            map.put(102, "Rahul");
            map.put(103, "Ramlal");
            //Navigable Map Methods.
                System.out.println(map.firstKey());
                System.out.println(map.lastKey());
                System.out.println(map.firstEntry());
                System.out.println(map.lastEntry());
                System.out.println(map.headMap(103));
                System.out.println(map.tailMap(102));
                System.out.println(map.subMap(101, 103));
            //Sorted Map methods.
                System.out.println(map.lowerKey(102 ));
                System.out.println(map.lowerEntry(102 ));
                System.out.println(map.floorKey(102 ));
                // all alternatives of treeset are here for the key and entry.
```

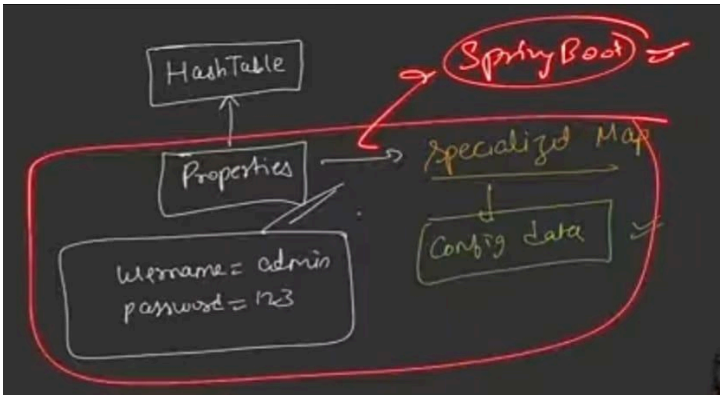
Jo bhi methods treeSet me ha wo sabhi respectively treeMap ke key & entry ke liya bhi use hote ha.

Other Map Classes

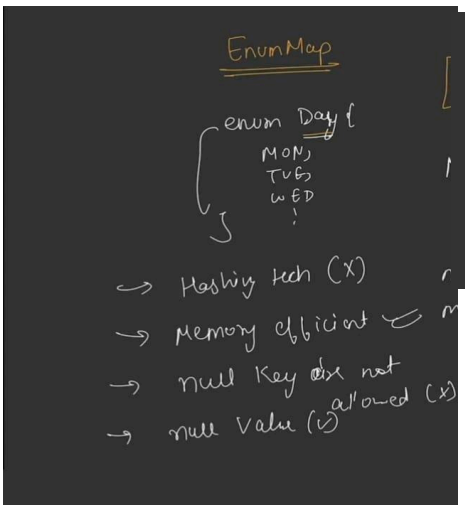
HashTables→

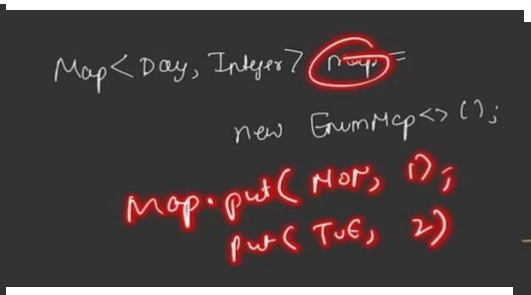


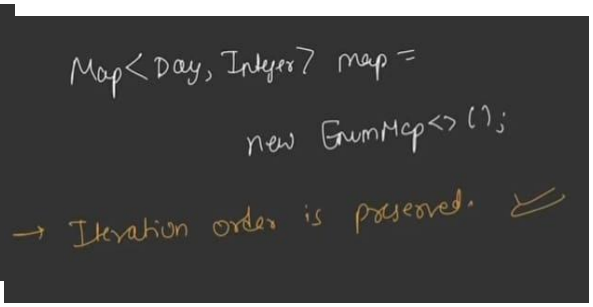
Properties→



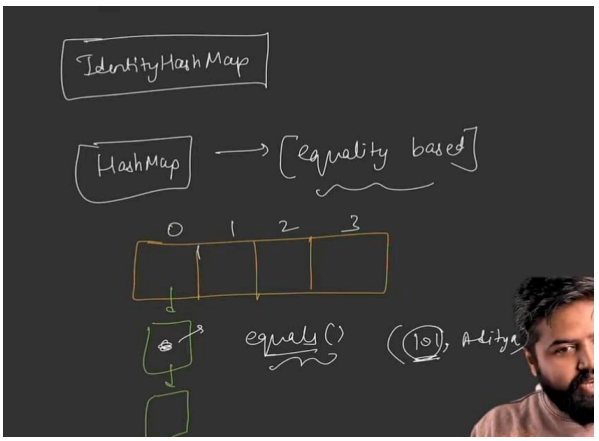
Enum Map→

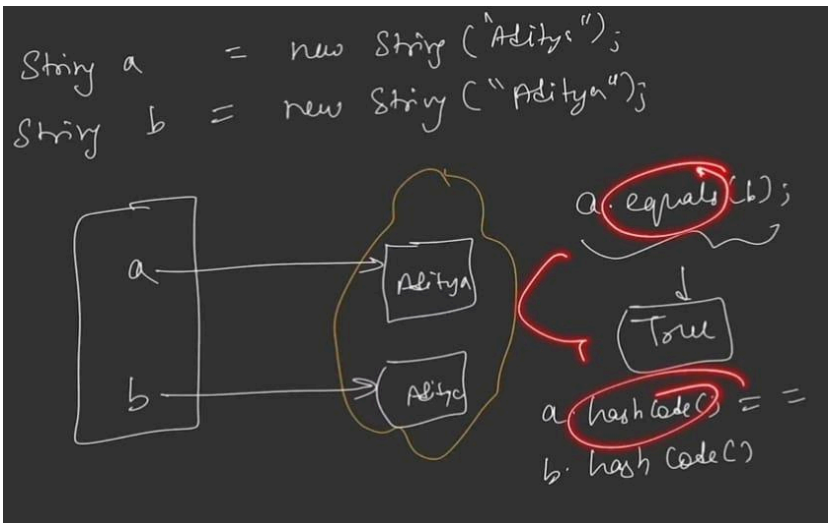






Identity HashMap→







Weak HashMap→

